

```

im = imread('EdgeImage.jpg');
%figure, imshow(im);

figure,
im2= im2double(im);
subplot(2, 2, 1),imshow(im2);
title('Iam2Double');

im3= rgb2gray(im2);
subplot(2, 2, 2),imshow(im3);
title('Rgb2Gray');

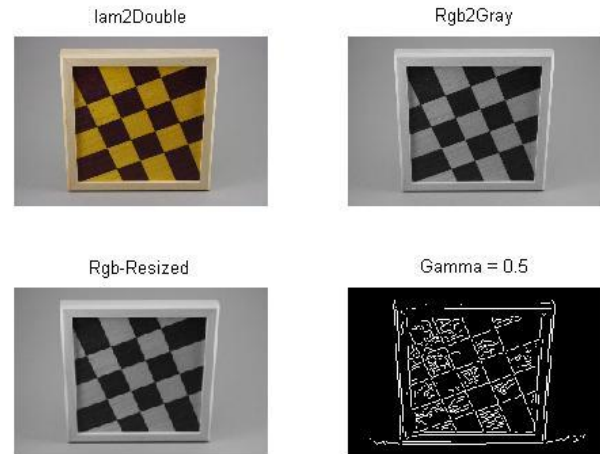
im4= imresize(im3, [167 250]);
subplot(2, 2, 3),imshow(im4);
title('Rgb-Resized');

im5=edge(im4, 'canny', [],0.5);
subplot(2, 2, 4),imshow(im5);
title('Gamma = 0.5');

suptitle('Transforming Image to
Find Edges');

```

Transforming Image to Find Edges



```

i=0.5;

figure,
for i=1:4,

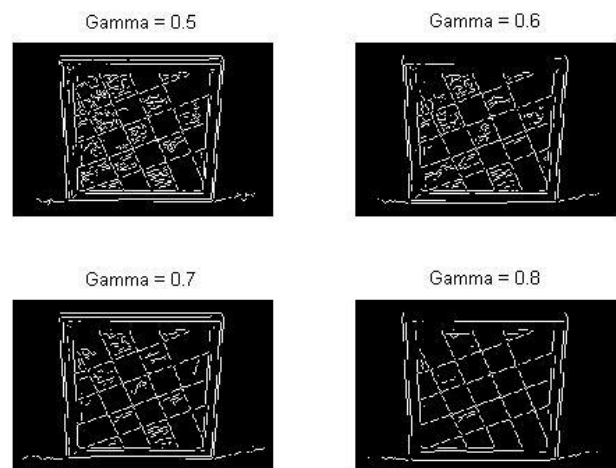
    imm=edge(im4, 'canny', [],0.4 +
i./10);
    subplot(2, 2, i),imshow(imm);

    if i==1
        title('Gamma = 0.5');
    end;

    if i==2
        title('Gamma = 0.6');
    end;
    if i==3
        title('Gamma = 0.7');
    end;
    if i==4
        title('Gamma = 0.8');
    end;
end;
suptitle('Finding Edges of an Image Gamma 0.5-0.8');

```

Finding Edges of an Image Gamma 0.5-0.8



Loading image in Matlab and assigning it to a variable named im. Then, the image is converted from to double follow by grayscale to resize it and finally apply the canny edge detection function. The images above show the first set of four pictures, this is an example of transformation from the original picture to the canny edge detection. And, the following set of four pictures shows different images whose edges vary due to a different gamma.

```

[iHeight, iWidth] = size(E);
distMax = round(sqrt(iHeight^2 + iWidth^2));

theta = -90:1:89;
H= 0;
rho= 0;
d= 0;
H = zeros(length(rho), length(theta));

for ix= 1: iWidth
    for iy= 1:iHeight
        if E(iy, ix) ~= 0
            for iTheta = 1:length(theta)
                t = theta(iTheta)*pi/180;
                dist = ix*cos(t) + iy*sin(t);
                [d, iRho] = min(abs(rho-dist));
                if d < 2
                    H(iRho,iTheta) = H(iRho, iTheta) + 1;
                end;
            end;
        end;
    end;
end;

```



This is the Hough transform code which consists of three for loops nested to scan through the image to test every edge of x and y. Initially, each cell is set to zero. Then, for every non-background point in the image, it computes the distance from the origin to the line along a vector perpendicular to the line (rho.) This is calculated for every theta and it is rounded off to the nearest allowed row in SHT. That accumulator cell is to increment the cell in my array H by one corresponding to the angle theta.

The figure to the left is the image with the wave forms of the transformed imaged whose contrast is adjusted in order to have a better view of the waveforms behaviors.

Experimenting

```
i=0.5;
figure,
for i=1:4,
    imm=edge(im4, 'canny', [], 0.4 + i./10);
    subplot(2, 2, i), imshow(imm);

    bw = edge(imm, 'canny', [], 2);
    imshow(bw);

    peaks = houghpeaks(H, 50);
    lines = houghlines(bw, theta, rho, peaks);
    imshow(bw)
    hold on

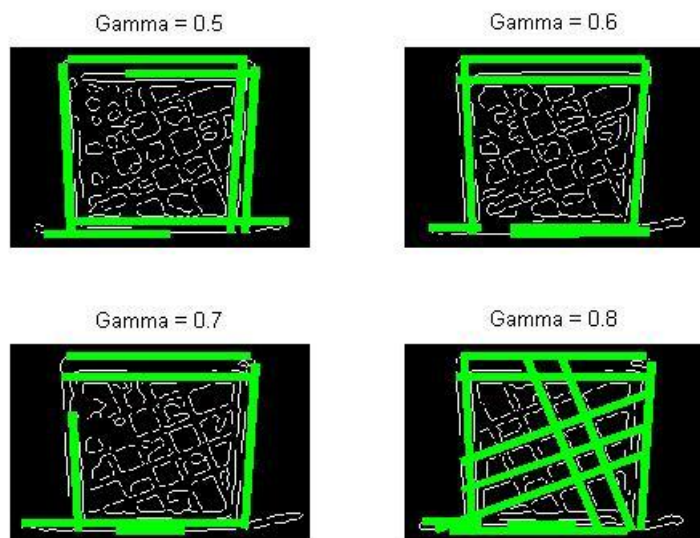
    for k = 1:numel(lines)
        x1 = lines(k).point1(1);
        y1 = lines(k).point1(2);
        x2 = lines(k).point2(1);
        y2 = lines(k).point2(2);
        plot([x1 x2], [y1 y2], 'Color', 'g', 'LineWidth', 4)
    end
    hold off

    if i==1
        title('Gamma = 0.5');
    end;

    if i==2
        title('Gamma = 0.6');
    end;
    if i==3
        title('Gamma = 0.7');
    end;
    if i==4
        title('Gamma = 0.8');
    end;

end;
suptitle('Finding Edges of an Image Gamma 0.5-0.8');
```

Finding Edges of an Image Gamma 0.5-0.8



Finding the edges of an image by using canny functions, this transforms the image with Hough and to draw its respected lines for gammas that vary from 0.5 to 0.8. As we can observe, in this set of pictures the gamma 0.8 is giving a better output since the Hough transformation seems to better recognize the edges at this level.

```

figure,
for x=1:4,

    imm=edge(im4,'canny',[],0.8 + x./10);
    subplot(2, 2, x),imshow(imm);

    bw = edge(imm, 'canny', [], 2);
    imshow(bw);

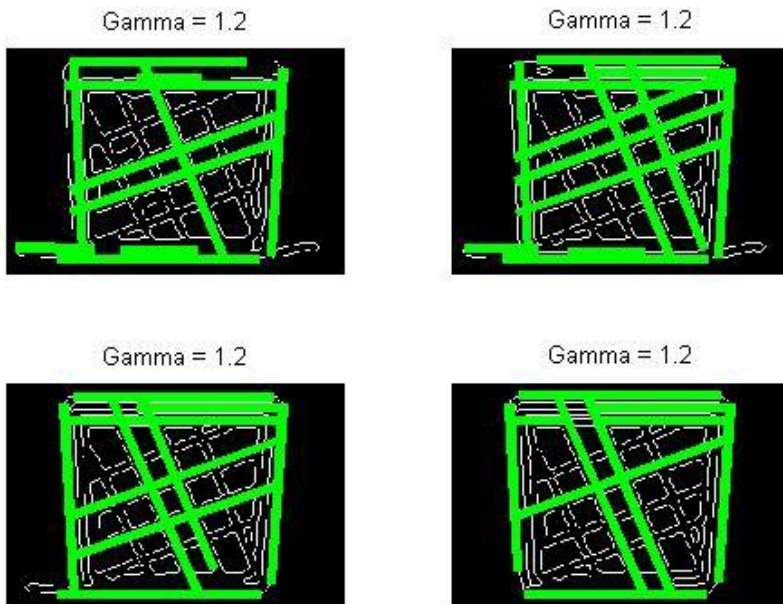
    peaks = houghpeaks(H,50);
    lines = houghlines(bw,theta,rho,peaks);
    imshow(bw)
    hold on

    for k = 1:numel(lines)
        x1 = lines(k).point1(1);
        y1 = lines(k).point1(2);
        x2 = lines(k).point2(1);
        y2 = lines(k).point2(2);
        plot([x1 x2],[y1 y2], 'Color','g','LineWidth', 4)
    end
    hold off

    if i==1
        title('Gamma = 0.9');
    end;
    if i==2
        title('Gamma = 1.0');
    end;
    if i==3
        title('Gamma = 1.1');
    end;
    if i==4
        title('Gamma = 1.2');
    end;
end;
suptitle('Finding Edges of an Image Gamma 0.9-1.2');

```

Finding Edges of an Image Gamma 0.9-1.2



This is another test, similar to the one above. The only difference is in the gamma since this is varying from 0.9 to 1.2. As we can observe, in this set of pictures the gamma 1.2 is giving a better output since the Hough transformation seems to better recognize the edges at this level.

```

figure,
for j=1:4,
    G= rgb2gray(im);
    G= imresize(G, [167 250]);
    E=edge(G, 'canny', 0 + j./10);

    peaks = houghpeaks(H, 50, 'Threshold', 30);
    I= imresize(im, [167 250]);
    subplot(2, 2, j), imshow(I); hold on;

    lines = houghlines(E, theta, rho, peaks, 'FillGap', 5, 'MinLength', 15);
    for k = 1:numel(lines)
        x1 = lines(k).point1(1);
        y1 = lines(k).point1(2);
        x2 = lines(k).point2(1);
        y2 = lines(k).point2(2);
        plot([x1 x2], [y1 y2], 'Color', 'b', 'LineWidth', 2)
    end

    if j==1
        title('Gamma = 0.1');
    end;

    if j==2
        title('Gamma = 0.2');
    end;

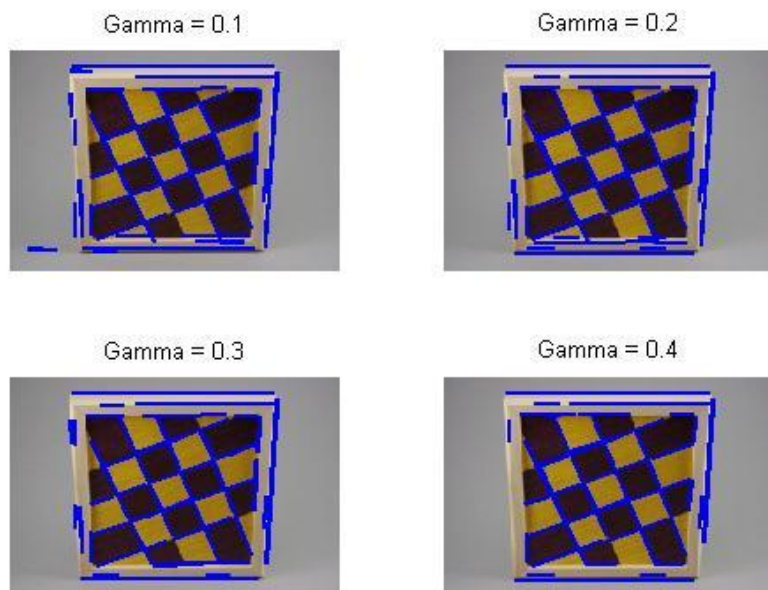
    if j==3
        title('Gamma = 0.3');
    end;

    if j==4
        title('Gamma = 0.4');
    end;

end; hold off
suptitle('Transforming Color Image to Find Edges');

```

Transforming Color Image to Find Edges



In this case I am trying the same approach as I did previously. Nevertheless, the image was not converted to the grayscale but only to a double scale. As a result, we can observe, in this set of pictures the gamma 0.1 is giving a better output since the Hough transformation seems to better recognize the edges at this level.

Write up

```
[iHeight, iWidth] = size(E);  
distMax = round(sqrt(iHeight^2 + iWidth^2));
```

This computes the maximum distance which is the diagonal distance.

```
theta = -90:1:89;  
H= 0;  
rho= 0;  
d= 0;  
H = zeros(length(rho), length(theta));  
  
for ix= 1: iWidth  
    for iy= 1:iHeight  
        if E(iy, ix) ~= 0  
            for iTheta = 1:length(theta)  
                t = theta(iTheta)*pi/180;
```

This creates an array of rho values. This initializes the parameters array to zero.

This creates an array of theta values from -90 to +89.

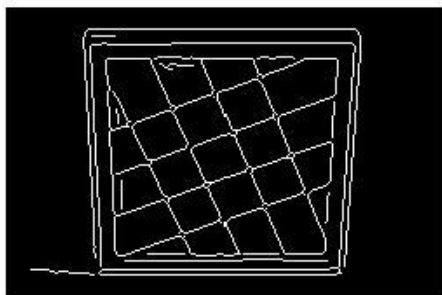
This scans through the image point by point looking for .0 edge point. If we find one, we scan through the values of Theta.

```
            dist = ix*cos(t) + iy*sin(t);  
  
            [d, iRho] = min(abs(rho-dist));  
  
                if d < 2  
                    H(iRho,iTheta) = H(iRho, iTheta) + 1;  
                end;  
            end;  
        end;  
    end;  
end;
```

Theta is an array of actual degree values, so theta is actually an array that looks like -90, 89, ... +89 - 1, 2, 180. Given a value by theta we can compute or retrieve the value of data corresponding to that.

Use the value of theta to compute the distance rho and then figure out which cell in the parameter space the actual value of data in the rho corresponds to.

```
imshow(im);  
G= rgb2gray(im);  
G= imresize(G, [167 250]);  
E=edge(G, 'canny', 0.1);  
figure, imshow(E);  
figure, imshow(H, []);  
  
peaks = houghpeaks(H, 50, 'Threshold', 30);  
I= imresize(im, [167 250]);  
figure, imshow(I, []), hold on
```



```

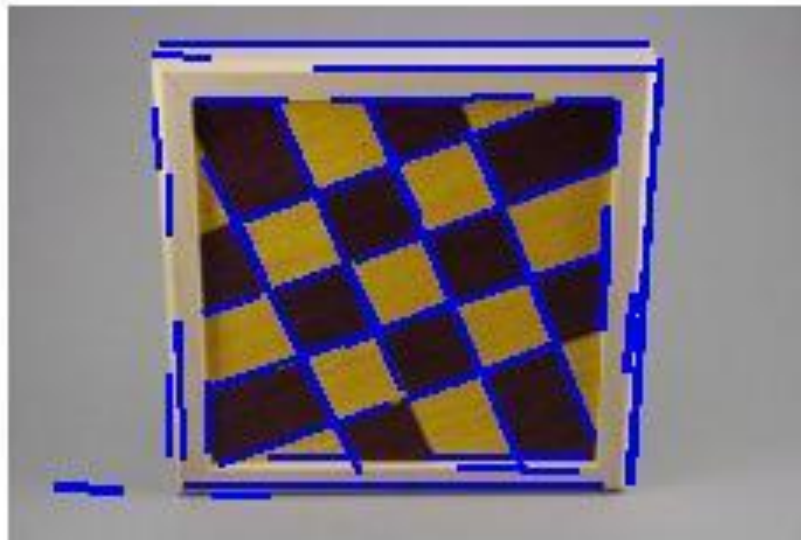
lines =
houghlines(E,theta,rho,peaks,'FillGap',5,'MinLength',15);
for k = 1:numel(lines)
    x1 = lines(k).point1(1);
    y1 = lines(k).point1(2);
    x2 = lines(k).point2(1);
    y2 = lines(k).point2(2);
    plot([x1 x2],[y1 y2],'Color','b','LineWidth', 2)
end
hold off

suptitle('Transforming Color Image to Find Edges');

```



Transforming Color Image to Find Edges



Overall, Hough transformation along with the canny edge detection got to recognize most of the edges of the picture by adjusting the gamma decimal value in the function. However, some edges of the picture were not recognized or detected by the program. This was expected since the Hough transformation as well as the canny edge detection algorithm is not a hundred per cent accurate because of the fact that there is always some noise intimating during or along the process of the transformation . This was proved by the using the “ imtools” from Matlab and showing the waveforms of the image. This showed how every waveform is behaving, and by adjusting the contrast of the graph it was a lot easier to observe how the noise cause some waveforms to intimate one another.